

COTVD: A function-level vulnerability detection framework using chain-of-thought reasoning with large language models

Yinan Chen^a, Xiangping Chen^c, Yuan Huang^{a,*}, Changlin Yang^a, Lei Yun^{a,b}

^a School of Software Engineering, Sun Yat-sen University, Zhuhai, Guangdong, China

^b Key Laboratory Ministry of Industry and Information Technology, China Electronic Product Reliability and Environment Testing Research Institute, Guangzhou, Guangdong, China

^c School of Journalism and Communication, Sun Yat-sen University, Guangzhou, Guangdong, China

ARTICLE INFO

Dataset link: <https://github.com/AFE23-u/CoTVD>

Keywords:

Vulnerability detection
Large language models
Chain of thought
Security risk analysis

ABSTRACT

Context: With the growth of open-source vulnerability databases, deep learning methods have been widely applied for vulnerability detection. However, most existing approaches focus on coarse-grained detection, predicting only whether a sample contains a vulnerability without providing detailed explanations.

Objective: This study aims to propose a large language model (LLM)-based method for function-level vulnerability detection and analysis, providing detailed insights into detected vulnerabilities.

Methods: We introduce CoTVD, which leverages the reasoning capabilities of LLMs through the Chain of Thought (CoT) approach. An instructive prompt strategy was designed to guide the model in analyzing data and control dependencies related to library function calls, enhancing detection flexibility and accuracy. CoTVD was evaluated on multiple LLMs, including GPT-4o-128K, GPT-3.5-Turbo, Gemini-1.5-Pro, Claude-3.5-Sonnet, Llama-3.1-405B, Qwen2-72B-Instruct-T, and DeepSeek-67B-T.

Results: Experimental results show that CoTVD based on GPT-4o-128K outperforms other models, achieving a recall rate of 94.77%. CoTVD, however, generates a relatively high number of false positives due to its sensitivity to potential security risks, which include not only traditional vulnerabilities but also risks such as information leakage. The study further refines existing dataset labels into finer granularity. In a human evaluation with 10 experts on 200 samples, each participant identified, on average, 1.7 additional vulnerability samples and 1.1 lines of risky code when assisted by CoTVD, demonstrating its effectiveness in real-world scenarios.

Conclusion: CoTVD enables function-level vulnerability detection with detailed analysis and proves effective in identifying vulnerabilities and code risks in practice, offering a novel approach for vulnerability detection and security analysis.

1. Introduction

Even experienced developers often make mistakes during software development, resulting in software vulnerabilities [1–4]. These vulnerabilities pose serious software security and quality threats, potentially resulting in system crashes, data leaks, or even major security incidents. With the widespread use of open-source code and increased code reuse, the spread of vulnerabilities has become even more severe, making effective vulnerability detection and repair critical challenges in software development and security [5].

Current deep learning-based vulnerability detection methods [6–9] automatically learn vulnerability-related patterns from large-scale datasets. These methods typically adopt sequence models (e.g., LSTM [10], Transformer [11]) or graph neural networks (GNNs) to represent

code semantics, such as Devign [12] and ReVeal [13]. While these approaches have demonstrated strong detection performance, their outputs are usually limited to binary labels or line-level importance scores. The resulting explanations are often post-hoc and descriptive, lacking explicit reasoning about how vulnerabilities arise from the interaction of data flow and control flow. Consequently, developers must manually reconstruct the underlying causes of vulnerabilities, which significantly increases the analysis burden and limits practical usability.

Recent advances in large language models (LLMs), exemplified by ChatGPT [14], offer new opportunities to address this limitation. Unlike traditional deep learning models, LLMs possess inherent reasoning capabilities and can generate coherent natural-language explanations when properly guided. Motivated by this observation, we

* Corresponding author.

E-mail address: huangyuan5@mail.sysu.edu.cn (Y. Huang).

<https://doi.org/10.1016/j.infsof.2026.108043>

Received 21 September 2025; Received in revised form 21 January 2026; Accepted 22 January 2026

Available online 3 February 2026

0950-5849/© 2026 Elsevier B.V. All rights are reserved, including those for text and data mining, AI training, and similar technologies.

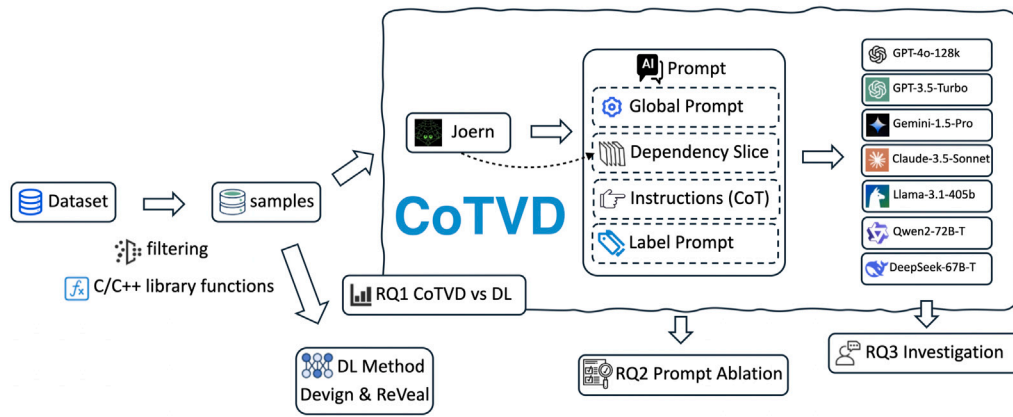


Fig. 1. The overview of our research.

propose CoTVD, a vulnerability detection framework that explicitly leverages Chain-of-Thought (CoT) reasoning [15] to perform structured, dependency-aware analysis. Rather than relying on post-hoc explanations, CoTVD guides the LLM to reason step by step over data dependencies and control dependencies [16], enabling the model to simulate the analytical process of human security experts. Through this reasoning-driven design, CoTVD not only determines whether a vulnerability exists, but also explains why it occurs by tracing unsafe data propagation paths and control conditions that lead to security-sensitive operations.

Fig. 1 illustrates the overall research framework of CoTVD. We focus on C/C++ code involving security-sensitive library functions and employ Joern [17] to extract compact dependency slices that capture critical data and control flow information. These slices are incorporated into a carefully designed prompt consisting of five components: a global prompt, source code context, dependency slice, reasoning instructions, and a label prompt. Based on this design, we implement CoTVD across multiple LLMs and evaluate its vulnerability detection performance in comparison with deep learning-based methods (RQ1). We further conduct ablation studies to analyze the contribution of different prompt components to reasoning quality and detection performance (RQ2). Finally, we assess the practical effectiveness of CoTVD in real-world scenarios (RQ3).

CoTVD has been evaluated on multiple LLMs, including GPT-4o-128K, GPT-3.5-Turbo, Gemini-1.5-Pro [18], Claude-3.5-Sonnet [19], Llama-3.1-405B [20], Qwen2-72B-Instruct-T [21] and DeepSeek-67B-T [22]. The experiment shows that the CoTVD based on GPT-4o-128K performs the best and achieves a recall rate of 94.77%. We also discovered that CoTVD produced a significant number of false positives. The analysis shows that CoTVD tends to mark any sample with potential security risks as positive, and here the security risks are not limited to traditional risks that lead to vulnerabilities, but also include risks such as information leakage. Finally, we further invited 10 developers to detect a total of 200 vulnerability samples. The results showed that, with the assistance of CoTVD, each developer was able to identify an average of 1.7 more vulnerable samples and detect an additional 1.1 lines of risky code compared to when they worked without CoTVD's support. In summary, the main contributions of this paper are as follows:

1. CoTVD adopts a carefully designed prompt strategy, which enhances the flexibility and accuracy of LLMs in analyzing data and control dependencies, enabling LLMs to more effectively capture various risks and vulnerabilities in the code.
2. CoTVD's fine-grained analysis provides developers with detailed explanations of the causes of vulnerabilities, helping them better understand and resolve these issues.

3. CoTVD is highly sensitive to security risks in the code, helping developers identify potential risks and defects in code design. At the same time, we also revealed the shortcomings of the existing dataset labels, especially in the automatic labeling method based on commits: code in commits that developers have not modified does not necessarily mean that the code is free of security risks. The security risks here may not be limited to traditional security risks that lead to vulnerabilities, but also include risks such as information leakage.

We have open-sourced CoTVD's slice extraction scripts, prompt construction methods, and related experimental results to support further research. These resources can be accessed at GitHub <https://github.com/AFE23-u/CoTVD>. The remainder of this paper is structured as follows: Section 2 provides an overview of existing vulnerability detection methods and explores the use of LLMs in software engineering; Section 3 details the implementation of CoTVD; Section 4 discusses our experimental design, presents the analysis of results; Section 5 gives a discussion; Section 6 covers the potential threats to the validity of CoTVD; and finally, Section 7 concludes with a summary of our research and suggestions for future work.

2. Related work

2.1. Vulnerability detection

The existing vulnerability detection methods can be primarily categorized into rule-based and deep learning-based approaches. Rule-based tools, such as Flawfinder [23] and Checkmarx [24], rely on rules defined by human experts to identify potential security vulnerabilities through pattern matching. While effective in specific contexts, these methods often produce high false positive rates, leading software maintainers to spend significant time validating false positives. In contrast, deep learning-based methods automatically learn vulnerability patterns from large datasets to detect vulnerabilities in code [25], functions, or code snippets [26,27]. These approaches typically treat source code as a sequence of tokens and employ sequence models like LSTM [10] or Transformer for classification. For example, VulDeePecker [26] extracts data dependencies from C/C++ library function calls, constructs code gadgets, and uses LSTM for vulnerability detection. SySeVR [27] further enhances VulDeePecker by incorporating additional syntactic features and control dependencies, improving detection performance and surpassing rule-based tools like Flawfinder and Checkmarx. Devign [12] introduces a novel graph neural network model that effectively identifies vulnerabilities by learning rich code semantic representations. This model employs graph-level classification to encode source code functions into a composite graph

structure, significantly enhancing the accuracy of vulnerability detection in real open-source projects and establishing a new benchmark in the field. In contrast, ReVeal [13] systematically investigates the shortcomings of deep learning-based vulnerability detection, revealing that existing models and datasets experience significant performance drops in real-world scenarios, often failing to capture the actual causes of vulnerabilities. To address these issues, ReVeal proposes a new data collection framework and a configurable vulnerability prediction tool, resulting in substantial improvements in detection precision and recall. Fan et al. [28] proposed VDoTR, which introduces a tensor-based representation that integrates multiple code graph structures to comprehensively capture code features. They also designed CircleGGNN to enable a more direct and effective fusion of heterogeneous graph information for embedding node states.

Recently, with the emergence of large language models, several studies have begun to explore Transformer- and LLM-based approaches for vulnerability detection. With the rapid advancement of large language models and Transformer-based architectures, recent studies have explored their application to software vulnerability detection [29,30]. Several works leverage pre-trained language models primarily as semantic feature encoders for source code. For instance, comparative studies have evaluated different text and language representations, such as word2vec, fastText, and BERT, by embedding source code into vector representations and feeding them into downstream classifiers (e.g., LSTM) for vulnerability prediction, demonstrating that context-aware embeddings can significantly improve detection accuracy.

Building upon this idea, VulDeBERT [31] fine-tunes a pre-trained BERT model on vulnerable C/C++ code fragments and achieves substantial performance improvements over earlier deep learning approaches such as VulDeePecker. Similarly, other studies investigate transfer learning strategies using Transformer-based models [32] such as CodeBERT and CodeGPT, exploring fine-tuning, embedding extraction, and hybrid training schemes to enhance vulnerability prediction performance. These works highlight the effectiveness of pre-trained language models in capturing syntactic and semantic patterns associated with vulnerable code.

Despite their promising detection performance, most existing LLM- or Transformer-based approaches primarily focus on improving classification accuracy and still treat vulnerability detection as a black-box prediction task. The model outputs are typically limited to vulnerability labels or confidence scores, while the underlying reasoning process remains implicit. Although some empirical studies have analyzed model behaviors and feature importance to improve interpretability, they do not provide explicit, human-readable explanations of why a vulnerability exists in a given code fragment.

In contrast, our work focuses on leveraging the reasoning capabilities of LLMs to perform structured, dependency-aware analysis and to generate natural-language explanations that explicitly trace the root causes of vulnerabilities through data-flow and control-flow reasoning, thereby addressing both detection performance and practical interpretability for developers.

2.2. LLM applications in software engineering

With the rise of LLMs, many researchers have conducted in-depth studies in recent years. Ma et al. [33] evaluated the performance of ChatGPT across various subdomains of software engineering [34–38]. Chen et al. [39] focused on several common vulnerabilities in the field of smart contracts, assessing ChatGPT’s effectiveness and limitations in detecting smart contract vulnerabilities, analyzing the reasons behind its false positives, and comparing it with other tools. Fan et al. [40] investigated whether automated program repair techniques could fix erroneous solutions generated by LLMs in LeetCode competitions. Zhang et al. [41] propose a methodology leveraging LLMs to identify inconsistencies within code comments. They utilize the natural language processing capabilities of LLMs, such as GPT-4, to

interpret and analyze design constraints embedded in code comments. By combining program analysis with LLM-driven comprehension, their approach detects instances where code comments diverge from actual code behavior. UniLog [42] is another application that uses LLMs to automatically generate logging through contextual learning, selecting log locations, determining verbosity, and generating log messages. Cheng et al. [43] systematically reviewed and expanded the concept of the “AI chain”, integrating best practices and principles accumulated over decades in the field of software engineering into AI chain engineering. Their proposed tool, Prompt Sapper, embodies these AI chain engineering principles and patterns within the development process, enabling AI chain engineers to create prompt-based AI services through chat-based requirements analysis and visual programming.

To leverage the reasoning and generative capabilities of LLMs, researchers have begun exploring their applications in more specific downstream tasks, designing tailored prompts to stimulate the model’s in-context learning (ICL [44,45]) and reasoning abilities [46–49]. For instance, TYPEGEN [50] introduced an automated type inference method that integrates knowledge from static analysis into LLMs through novel prompt designs to improve type inference results. TYPEGEN combines code slicing and type dependency analysis, guiding the model to generate type information through chain reasoning.

Inspired by these works, CoTVD combines static code analysis with the reasoning capabilities of LLMs, achieving function-level vulnerability detection through the chain-of-thought mechanism. In CoTVD, the LLM is guided to gradually analyze the data dependencies and control dependencies within functions, allowing for more accurate detection of potential vulnerabilities. This approach not only identifies existing security risks but also provides detailed analytical explanations, assisting developers in understanding and resolving issues in the code.

3. The design of CoTVD

We will detail the design of CoTVD in this section, including the dataset collection and processing methods, the library function call dependency extraction methods, and prompt components and intentions.

3.1. Dataset collection and processing

The dataset used in this study is sourced from Devign [12] and ReVeal [13], both of which are constructed from real-world C/C++ projects. Devign is collected from four large open-source C projects, including the Linux kernel, QEMU [51], Wireshark [52], and FFmpeg [53]. ReVeal provides a comprehensive real-world vulnerability dataset covering two widely used open-source projects, namely the Linux Debian kernel and Chromium.

Following prior vulnerability detection research, particularly VulDeePecker [26], we observe that a large proportion of real-world vulnerabilities arise during the invocation of security-sensitive C/C++ library or API functions. Such functions are often closely associated with common vulnerability types, including buffer overflows, memory corruption, and improper input handling. Typical examples include, but are not limited to, `getenv`, `strncpy`, and `malloc`, as shown in Listing 1.

Motivated by this observation, this study focuses on samples containing C/C++ library function calls and treats these calls as security-sensitive anchors for dependency analysis. This design choice provides clear and well-defined slicing criteria, enabling precise extraction of data-flow and control-flow dependencies using static analysis. Moreover, centering the analysis around library function calls allows CoTVD to perform structured, step-by-step reasoning over vulnerability-prone execution paths, which aligns naturally with the Chain-of-Thought-guided analysis adopted in our framework.

Listing 1: C/C++ library function calls.

```

1 // CWE-119
2 cin, getenv, getenvs, wgetenv, wgetenvs, catgets,
  gets, getchar,getc, getch, getche, kbhit, stdin,
  getdlogtext, getpass, scanf, fscanf, vsprintf,
  vscanf, istream.get, istream.getline, istream.
  peek, istream.read*, istream.putback, streambuf
  .sbumpc, streambuf.sgetc, streambuf.sgetn,
  streambuf.snextc, streambuf.sputbackc,
  SendMessage, SendMessageCallback,
  SendNotifyMessage, PostMessage,
  PostThreadMessage, recv, recvfrom, Receive,
  ReceiveFrom, ReceiveFromEx, Socket.Receive*,
  memcpy, wmemcpy, memccpy, memmove, wmemmove,
  memset, wmemset, memcmp, wmemcmp, memchr,
  wmemchr, strncpy, strncpy*, lstrncpy, tcscpy*,
  mbsnbcpy*, wcsncpy*, wcsncpy, strncat, strncat*,
  mbsncat*, wcsncat*, bcopy, strcpy, lstrcpy,
  wcscpy, tcscpy, mbscpy, CopyMemory, strcat,
  lstrcat, strlen, strchr, strcmp, strcoll,
  strcspn, strerror, strlen, strpbrk, strrchr,
  strspn, strstr, strtok, strxfrm, readlink, fgets,
  , sscanf, swscanf, sscanf, swscanf, printf,
  vprintf, swprintf, vsprintf, asprintf, vasprintf,
  , fprintf, sprint, snprintf, sprintf*,
  snwprintf*, vsnprintf, CString.Format, CString.
  FormatV, CString.FormatMessage, CStringT.Format,
  CStringT.FormatV, CStringT.FormatMessage,
  CStringT.FormatMessageV, syslog, malloc,
  Winmain, GetRawInput*, GetComboBoxInfo,
  GetWindowText, GetKeyNameText, Dde*, GetFileMUI
  *, GetLocaleInfo*, GetString*, GetCursor*,
  GetScroll*, GetDlgItem*, GetMenuItem*
3 // CWE-399
4 free, delete, new, malloc, realloc, calloc, alloca,
  strdup, asprintf, vsprintf, vasprintf, sprintf,
  snprintf, snwprintf, vsnprintf

```

We will introduce how to extract the data dependencies and control dependencies related to these library function calls in the following.

3.2. Function-level dependency extraction

We utilize Joern [17], a query-based static analysis framework built on Code Property Graphs (CPGs), to extract data and control dependencies associated with security-sensitive C/C++ library and API function calls. Joern unifies abstract syntax trees, control-flow graphs, and data-flow graphs, enabling precise program dependence analysis. Based on Joern, we construct compact dependency slices that capture the critical execution paths relevant to vulnerability detection while excluding unrelated code.

Algorithm 1 illustrates the procedure for extracting function-level dependency slices. Given a function-level C/C++ code sample, the algorithm outputs a dependency slice that represents the security-critical data-flow and control-flow context associated with sensitive function calls.

As shown in Algorithm 1, the input to the dependency extraction process is a function-level C/C++ code sample, and the output is a compact dependency slice associated with security-sensitive library function calls. The detailed procedure is as follows.

First, an empty set S is initialized to store the extracted dependency statements. Next, we construct a predefined list L of security-sensitive C/C++ library and API functions, such as `printf` and `getenv`, which are commonly associated with potential security risks (as shown in Listing 1). All function calls in the input code sample F that match entries in L are identified as target calls.

For each identified security-sensitive function call, we extract its argument variables and perform dependency analysis using Joern. Specifically, Joern is used to construct a program dependence graph (PDG)

Algorithm 1: Dependency Slice Extraction for Security-Sensitive Function Calls

Input: Function-level code sample F

Output: Dependency slice S

begin

$S \leftarrow \emptyset$;

Load predefined list of security-sensitive C/C++ library/API functions L ;

Identify all function calls $C \subset F$ such that $c \in L$;

foreach security-sensitive function call $c \in C$ **do**

Extract argument variables V_c of c ;

foreach variable $v \in V_c$ **do**

Use Joern to extract backward data dependency statements influencing v ;

Use Joern to extract control dependency statements governing the execution of c ;

Add all extracted data and control dependency statements to S ;

Order statements in S according to program execution order;

return S ;

for the enclosing method. Through Joern's built-in queries (e.g., `dot-Pdg`), we extract backward data dependency statements that transitively influence the argument variables, as well as control dependency statements that determine whether and under what conditions the function call is executed. This process effectively performs backward dependency slicing from the security-sensitive function call, capturing both data-flow influences and control-flow constraints.

All extracted dependency statements are then aggregated and ordered according to their execution sequence in the program, forming a dependency slice that represents the critical execution path related to the target function call. This slice preserves essential semantic context for vulnerability analysis while significantly reducing irrelevant code.

To ensure transparency and reproducibility, the Joern-based dependency extraction scripts used in this work have been fully open-sourced. Given a compiled CPG and a list of target function names, these scripts automatically extract and serialize the corresponding dependency slices, enabling the proposed method to be easily reproduced and extended.

3.3. Prompt design

We carefully designed a prompt consisting of five components to effectively leverage the reasoning capabilities of LLMs for in-depth vulnerability analysis on extracted library function call dependency slices. The core objective of CoTVD is not merely to provide additional contextual information, but to explicitly guide the model to perform structured, step-by-step reasoning over security-critical code paths, thereby simulating the analytical process of human security experts.

In CoTVD, the Chain-of-Thought (CoT) is operationalized through explicit reasoning instructions in the prompt, rather than being implicitly assumed from the model's output alone. As illustrated in Fig. 1 and Listing 2, the complete prompt consists of four components: global prompt, library function call dependency slice, instructions, and label prompt. Among these, the dependency slice provides the necessary structural information, while the instructions explicitly enforce the Chain-of-Thought reasoning process. The complete prompt is shown in Listing 2. We next describe the design rationale of each component in detail.

3.3.1. Design intent

Global Prompt. As shown in the second line of Listing 2, the role of “code auditor” is first assigned to the LLM to analyze code and identify potential vulnerabilities. This role specification establishes a global context for interaction, ensuring that all subsequent analyses are based on this premise, making the LLM focus on the task of vulnerability detection in C/C++ code, thereby improving detection accuracy.

Dependency Slice. To enable the LLM to fully utilize the previously extracted data dependencies and control dependencies related to library function calls, we explicitly include the extracted dependencies slice dependencies in the prompt, as shown in line 4 of Listing 2. This prompt instructs the LLM to consider the data flow between these library function calls and how the control flow coordinates their execution.

Listing 2: CoTVD prompt.

```

1 # Global Prompt
2 You are a code auditor tasked with reviewing the
   following C/C++ code. Your goal is to analyze the
   code and identify any vulnerabilities or security
   flaws.
3 # Code slice extracted: {dependencies}
4 # Instructions
5 1. Analyze the provided code carefully. Focus on the
   function calls {func_list} in the slice and
   consider how they interact with the surrounding
   code.
6 2. Pay attention to how data is passed to and from the
   function calls and how the control flow is
   managed around them.
7 # Label Prompt
8 You must first thoroughly analyze the code. Only after
   completing the analysis should you provide your
   detection result:
9 - If no vulnerabilities are found, output: 'Label:0'
10 - If vulnerabilities are found, output: 'Label:1'

```

Instructions. The instructions are specifically designed to implement the Chain-of-Thought reasoning in CoTVD by guiding the LLM through a multi-step vulnerability analysis process, as shown in lines 4–6 of Listing 2. First, the instructions require the LLM to identify and focus on security-sensitive library function calls within the dependency slice `func_list`. Next, the model is guided to analyze the data-flow relationships among these function calls, trace the propagation of critical parameters, and examine how control-flow dependencies govern their execution order.

Overall, these instructions explicitly enforce a chain-structured reasoning process over code dependencies, which constitutes the Chain-of-Thought in CoTVD. By decomposing vulnerability detection into a sequence of interpretable analytical steps, the instructions enable the LLM to simulate human expert reasoning, thereby improving both detection accuracy and the interpretability of the generated analysis.

Label Prompt. The design of the label prompt is as shown in lines 10–13 of Listing 2. The LLM is explicitly instructed to analyze the code before providing detection results. This setup is intended to ensure that the LLM has conducted sufficient analysis and thought before making a conclusion. Label prompts help improve the LLM’s analytical results and detection label consistency, and also facilitate review by human developers.

3.3.2. Summary

CoTVD aims to promote a structured and comprehensive analysis process for LLMs on vulnerabilities, reflecting the vulnerability detection methods of human experts. The global prompt establishes the role and purpose of the LLM, aligning its focus with the C/C++ vulnerability detection task. Source code and dependency slice provide LLM with

the necessary complete context and insights into specific dependencies. Instructions provide the model with a set of systematic guidelines, prompting it to analyze data flow, control flow, and external dependencies. Finally, label prompts require the model to comprehensively analyze the code before outputting the label.

4. Experiment design and evaluation

In this section, we will introduce the experimental design of CoTVD, including dataset statistics, LLMs and deep learning baselines for evaluation, software and hardware environments, evaluation metrics, and the research questions proposed.

4.1. Dataset and experiment setup

Dataset. The dataset used in the experiment comes from Devign and Reveal, both of which are C/C++ datasets, as introduced in Section 3.1. Since CoTVD focuses on detecting vulnerabilities related to C/C++ library function calls, we have removed samples that do not contain such function calls. The statistical information of the dataset is displayed in Table 1.

Finally, we collected 3435 positive samples and 6018 negative samples. Due to LLMs’ inference cost limitations, we mixed these samples and randomly selected 1/10 of them as the test set, resulting in 344 positive samples and 602 negative samples.

Experiment Environment. All experiments were conducted under a unified hardware and software configuration to ensure fair comparison across models. The server is equipped with an Intel(R) Xeon(R) Gold 6348 CPU and NVIDIA A800 (80 GB) GPUs. The software environment includes Ubuntu 20.04 LTS, Python 3.7, and Joern for static code dependency extraction. For all large language model inferences, we adopt a fixed sampling temperature of 0.7, which is commonly used as a balanced setting between output stability and reasoning diversity in LLM-based analysis tasks. Prior empirical studies have shown that varying the temperature within the typical range (e.g., 0.0–1.0) does not lead to statistically significant differences in task performance, while primarily affecting output variability [54]. To avoid bias toward a single model family and to assess the generality of our approach, we evaluate CoTVD on a diverse set of large language models. The selected discourse-level large language models include both commercial models—GPT-4o-128K (128 K context), GPT-3.5-Turbo (up to 16 K context), Claude-3.5-Sonnet (up to 200 K context), and Gemini-1.5-Pro (up to 1M context)—as well as state-of-the-art open-source models—Llama-3.1-405B [55] (up to 128 K context), Qwen2-72B-Instruct-T [56] (up to 128 K context), and DeepSeek-LLM-67B-T [57] (up to 16 K context). These models were chosen to represent a broad spectrum of reasoning capabilities, model scales, and deployment scenarios. By including both commercial and open-source LLMs with varying context window sizes, we aim to comprehensively evaluate the robustness, practicality, and scalability of CoTVD under different real-world settings.

4.2. Evaluation metrics

To comprehensively evaluate the performance of CoTVD and baselines, we used the following evaluation metrics [58]:

Number of Detected Positive Samples: Refers to the number of samples correctly identified as positive (vulnerable) by the model.

Precision: The proportion of correctly detected positive samples out of all samples classified as positive by the model. The formula is:

$$\text{Precision} = \frac{TP}{TP + FP}$$

where TP is the number of true positives, and FP is the number of false positives.

Recall: The proportion of actual positive samples that were correctly identified by the model. The formula is:

$$\text{Recall} = \frac{TP}{TP + FN}$$

Table 1

Statistics of the datasets: Vul stands for vulnerable (positive) samples, and Non-Vul for non-vulnerable (negative) samples.

Dataset	Vul	Non-Vul	Filtered Vul	Filtered Non-Vul
Devign	14 858	12 460	2703	2944
Reveal	2240	20 494	491	3315
Total	17 098	32 954	3435	6018
Selected for testing	–	–	344	602

Table 2

Experimental results of CoTVD based on different LLMs and deep learning baselines.

Model	Precision (%)	Recall (%)	F_1 -score	#Detected positive samples
GPT-4o-128K	39.09	94.77	0.553	326
GPT-3.5-Turbo	35.49	63.66	0.456	219
Gemini-1.5-Pro	38.11	88.08	0.532	303
Claude-3.5-Sonnet	34.04	69.77	0.458	240
Llama-3.1-405B	29.09	60.46	0.393	208
Qwen2-72B-Instruct-T	35.03	65.99	0.458	227
DeepSeek-67B-T	46.91	11.04	0.179	38
ReVeal	33.19	70.05	0.450	241
Devign	31.10	64.82	0.420	223
LineVul	53.30	56.97	0.551	196

where FN is the number of false negatives.

F_1 Score: The harmonic mean of precision and recall, providing a balanced measure of model performance. The formula is:

$$F_1 = 2 \cdot \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$$

4.3. Research questions

To comprehensively evaluate the effectiveness and practical value of CoTVD in vulnerability detection, we designed the following research questions. RQ1 aims to explore the effectiveness of CoTVD in different LLMs and compare its performance with traditional deep learning models. RQ2 aims to investigate the effectiveness of the CoTVD design, particularly the role of the “dependency slice” component. RQ3 will assess the interpretability of CoTVD and its application effects in real-world scenarios.

RQ1: How effective is CoTVD based on different LLMs, and how does them compare to traditional deep learning models?

RQ2: How do different components of CoTVD contribute to vulnerability detection performance?

RQ3: What is the practical value of CoTVD? Does it have explainability for vulnerability detection and help developers improve efficiency?

Next, we will present the experimental design and result analysis for each research question.

4.3.1. RQ1: How effective is CoTVD based on different LLMs, and how does it compare to traditional deep learning models?

We applied CoTVD on multiple LLMs to test 946 samples, including GPT-4o-128K, GPT-3.5-Turbo, Gemini-1.5-Pro, Claude-3.5-Sonnet, Llama-3.1-405B, Qwen2-72B-Instruct-T, and DeepSeek-LLM-67B-T. Additionally, we evaluated the performance of traditional deep learning methods Devign and ReVeal on the same test set for comparison. The experimental results are shown in Table 2.

As shown in Table 2, GPT-4o-128K-based CoTVD stands out among all models, achieving a F_1 score of 0.553 with a precision of 39.09% and a recall rate of 94.77%, detecting 326 positive samples. The suboptimal model, Gemini-1.5-Pro, detected 303 positive samples with a recall rate of 88.08%, ranking second in performance with a F_1 score of 0.532. Compared to GPT-4o-128K, although the precision is slightly lower, it still maintains a high recall rate, indicating its strong sensitivity to vulnerability samples. DeepSeek-67B-T performed the

worst, with the highest precision of 46.91% but a recall rate of only 11.04%, resulting in a F_1 score as low as 0.179, detecting 38 positive samples. This shows that although it can accurately detect some positive samples, a large number of vulnerable samples are missed, limiting its overall detection effectiveness.

Compared to deep learning methods, GPT-4o-128K-based CoTVD detects 85 and 103 more positive samples than ReVeal and Devign, respectively, with recall rates 24.72 and 29.95 percentage points higher. Similarly, CoTVD based on Gemini-1.5-Pro detects 62 and 80 more positive samples than ReVeal and Devign, respectively. This indicates that CoTVD significantly outperforms traditional deep learning methods in terms of recall rate. In addition to DeepSeek-67B-T, the performance of other CoTVD benchmark models is comparable to deep learning methods. In addition to graph-based deep learning baselines such as Devign and ReVeal, we further compare CoTVD with LineVul, a representative fine-tuned Transformer-based vulnerability detection model. As shown in Table 2, LineVul achieves the highest precision (53.30%) among all evaluated models; however, its recall (56.97%) is significantly lower than the GPT-4o based CoTVD.

In summary, GPT-4o-128K-based CoTVD showed the best detection performance in this experiment. Compared to deep learning models, CoTVD not only improved the detection accuracy but also provided more intuitive interpretability for vulnerabilities.

4.3.2. RQ2: How do different components of CoTVD contribute to vulnerability detection performance?

To systematically evaluate the contribution of each component in CoTVD, we conducted a comprehensive ablation study on the prompt design. Specifically, CoTVD consists of two key components: (1) the Dependency Slice, which provides security-sensitive code contexts based on data and control dependencies; and (2) the Chain-of-Thought (CoT) instructions, which explicitly guide the model to perform step-by-step reasoning over the dependency slices. To isolate the effect of each component, we constructed two ablation variants: (i) **GPT-4o-w/o-Slice**, where the dependency slice and its related prompt content are removed, and the model is only provided with the full source code; and (ii) **GPT-4o-w/o-CoT**, where the dependency slice is retained, but the explicit step-by-step reasoning instructions are removed.

All experiments were conducted using the best-performing GPT-4o-128K-based model identified in RQ1. The results of the ablation study are summarized in Table 3.

As shown in the table, the full CoTVD prompt achieves the best overall performance, with a precision of 39.09%, a recall of 94.77%,

Table 3
Ablation experiment on GPT-4o-128K based model.

Model	Precision (%)	Recall (%)	F_1 -score
GPT-4o-128K	39.09	94.77	0.553
GPT-4o-w/o-Slice	37.18	92.44	0.530
GPT-4o-w/o-CoT	36.56	94.48	0.527

and an F_1 -score of 0.553. Removing the dependency slice (GPT-4o-w/o-Slice) leads to a noticeable performance degradation across all metrics, indicating that structured dependency information plays a critical role in helping the model identify vulnerable execution paths.

More importantly, when the Chain-of-Thought instructions are removed while retaining the dependency slices (GPT-4o-w/o-CoT), the model also exhibits a clear decline in performance, particularly in precision and F_1 -score. Although the recall remains relatively high, the reduction in precision suggests that, without explicit step-by-step reasoning guidance, the model is more prone to producing false positives.

These results demonstrate that both the dependency slice and the Chain-of-Thought instructions are indispensable components of CoTVD. While dependency slices provide essential contextual information, the Chain-of-Thought instructions play a crucial role in guiding the model to systematically reason over this information. Together, they enable CoTVD to achieve not only improved detection performance but also significantly enhanced interpretability of vulnerability analysis results.

4.3.3. RQ3: What is the practical value of CoTVD? Does it have explainability for vulnerability detection and help developers improve efficiency?

In order to evaluate the effectiveness of CoTVD in real-world scenarios, we designed the following experiments. We selected 200 samples that were successfully detected by CoTVD, manually annotated their line numbers of vulnerabilities and randomly divided them into two groups. One group contains the analysis results from CoTVD, referred to as the “CoTVD group”, while the other group provides only the source code, referred to as the “source code group”. We invited 10 experienced human experts to participate in the vulnerability detection survey. They all have at least five years of C/C++ development experience. Each participant was assigned 10 samples from the CoTVD group and 10 samples from the source code group.

For each sample, participants were asked to determine whether a vulnerability existed. If they believed a vulnerability was present, they were further asked to indicate the exact line number where the vulnerability occurred. At the same time, we also recorded the time participants took each group of questionnaires. The questionnaire was designed as follows:

Source Code Group:

Please determine if there is a vulnerability in the source code. If so, please specify the line number.

[Source Code]

- Vulnerability found, specify the line number:
- No vulnerability

CoTVD Group:

Please combine the CoTVD analysis with the source code to determine if there is a vulnerability. If so, please specify the line number.

[Source Code]

[CoTVD Analysis]

- Vulnerability found, specify the line number:
- No vulnerability

Finally, we collected the results from the 10 participants regarding the number of vulnerability samples detected, the number of lines of vulnerabilities identified, and the average time required for the CoTVD group and the source code group. The results are shown in Table 4.

Table 4
Survey results: CoTVD group vs source code group.

Group	Vulnerabilities	Correct line	Average time (s)
Source Code	6.4	8.8	577
CoTVD	8.1	9.9	441.5

As shown in Table 4, in terms of the number of vulnerabilities detected, participants identified an average of 8.1 vulnerabilities in the CoTVD group, which is 1.7 more than the 6.4 vulnerabilities identified in the source code group. This indicates that CoTVD can help developers identify more vulnerable samples. Furthermore, regarding the number of lines identified for the vulnerabilities, participants performed better in the CoTVD group, correctly identifying an average of 9.9 lines compared to 8.8 lines in the source code group. This indicates that CoTVD also helps developers locate vulnerabilities.

Finally, participants spent less time in the CoTVD group (441.5 s) compared to the source code group (577 s). This outcome demonstrates that CoTVD not only enhances the effectiveness of vulnerability detection but also reduces the time developers need to complete detection tasks, thus improving the efficiency. These findings further confirm the effectiveness and practical value of CoTVD in vulnerability detection, making it a valuable tool for assisting developers in identifying and addressing security issues.

4.4. Experimental findings

The effectiveness of CoTVD in real-world vulnerability detection scenarios. Experiments have proved that the natural language analysis provided by CoTVD has a good effect in assisting developers to improve the coverage and accuracy of vulnerability detection. Specifically, participants identified an average of 8.1 vulnerabilities in the CoTVD group, 1.7 more vulnerabilities than in the source code group. In addition, participants’ average detection time on the CoTVD group was 441.5 s, which was significantly lower than the 577 s on the source code group, indicating that CoTVD’s natural language analysis significantly reduced the cognitive load on developers and improved detection efficiency. This natural language analysis provides developers with a comprehensive explanation of vulnerabilities, including data dependencies and control dependency analysis. Therefore, CoTVD effectively supports developers’ vulnerability detection task, improves the comprehensiveness and interpretability of vulnerability detection, thus demonstrating its practical value in real applications.

CoTVD can identify potential security risks and code design defects. CoTVD shows a clear advantage in identifying potential security risks, capable of effectively detecting potential risks and design defects in the code, such as information leakage. Although these risks may not be classified as traditional vulnerabilities (for example, errors that directly lead to system crashes or functional failures), they may still cause major security issues. In the detection process of CoTVD, if the code contains print statements that may leak sensitive data, insecure or incomplete error handling, or lack of checks for unknown elements, CoTVD will mark these samples as positive, thereby alerting developers to potential security risks. Experimental results show that this detection strategy enables developers to have a more comprehensive understanding of potential risks and design defects in the code, further enhancing security.

5. Discussion

5.1. Efficiency and practicality of dependency slicing

An important design consideration of CoTVD is the trade-off between analysis accuracy and practical efficiency when integrating static analysis tools with large language models. While dependency slicing

Table 5

Token reduction analysis with dependency slicing.

Metric	Full function	Dependency slice
Average input tokens	900.04	328.94

introduces additional preprocessing overhead through Joern, our empirical token-level analysis demonstrates that this overhead is well justified.

Specifically, by replacing full function bodies with dependency slices augmented by structured instructions, the average number of input tokens is reduced from 900.04 to 328.94 across 946 samples evaluated using GPT-4o, corresponding to an average reduction of 571.10 tokens (63.45%), as summarized in Table 5. This substantial reduction indicates that dependency slicing effectively compresses the input context while preserving security-critical semantic information.

From a practical perspective, this reduction has two important implications. First, by removing irrelevant code statements, dependency slicing mitigates the risk of LLM hallucination caused by distracting or semantically unrelated context. Second, the significantly reduced token footprint alleviates token budget constraints, enabling more scalable and cost-effective vulnerability analysis when deploying LLM-based approaches in real-world settings.

To evaluate scalability using open-source models, we measured inference latency on a deployment with $4 \times$ NVIDIA A80 (80 GB) GPUs. As shown in Table 6, the average inference time per sample is approximately 5.8 s for Qwen2-72B-Instruct-T and 5.2 s for DeepSeek-67B-T. While these models exhibit lower detection performance than commercial alternatives, their inference latency indicates that CoTVD can be integrated into CI/CD pipelines under batch or asynchronous execution settings.

Regarding the observed performance gap between commercial and open-source models, our analysis suggests that the limitation primarily lies in instruction-following fidelity and multi-step reasoning capability rather than context window size or input length. Since dependency slicing significantly compresses the input context, token limitations are unlikely to be the dominant factor. This observation highlights the importance of advanced reasoning capabilities for effectively leveraging structured prompts such as CoTVD, and suggests that future improvements in open-source LLMs may further narrow this gap.

Overall, these findings demonstrate that CoTVD achieves a balanced trade-off between detection effectiveness, interpretability, and practical deployability, making it suitable for both research and real-world vulnerability detection scenarios.

5.2. Example of interpretable vulnerability reasoning

To further highlight the interpretability advantage of CoTVD, we present a concrete example illustrating how the proposed approach produces a vulnerability prediction together with a human-readable reasoning process. Unlike conventional deep learning models that only output a binary label, CoTVD provides a step-by-step natural-language explanation that aligns with how security experts reason about vulnerabilities.

Listing 3: An example code slice analyzed by CoTVD.

```
void simplestring_addn(simplestring *target, const
char *source, int add_len) {
    if (target && source) {
        if (!target->str) {
            simplestring_init_str(target);
        }
        if (target->len + add_len + 1 > target->size) {
            newsize = newsize - (newsize target->str =
                (char *)realloc(target->str, newsize)
            );
        }
    }
}
```

```
target->size = target->str ? newsize : 0;
}
if (add_len) {
    memcpy(target->str + target->len, source,
        add_len);
    target->len += add_len;
}
}
```

Listing 4: Interpretable vulnerability reasoning generated by CoTVD.

```
Step 1: Identify security-sensitive function calls.
The function contains a call to memcpy, which is known
to be vulnerable to buffer overflow if the copy
length is not properly validated.=
Step 2: Analyze data dependencies.
The length parameter add_len is directly used as the
third argument of memcpy. Although a size check
exists, the actual allocated size depends on
newsize, whose correctness cannot be fully
verified within the slice.
Step 3: Analyze control dependencies.
The result of realloc is not explicitly validated
before memcpy is executed. If realloc fails,
target->size may become zero while the memory
copy is still performed.
Step 4: Vulnerability conclusion.
Due to the potential mismatch between allocated
buffer size and the copy length, this code may
lead to a heap-based buffer overflow.
Label: 1
```

This example demonstrates that CoTVD does not merely predict whether a function is vulnerable, but also explains why a vulnerability may occur. The reasoning explicitly traces security-relevant library calls, data dependencies, and control-flow decisions, producing an explanation that is easy for developers to understand and verify. Such interpretability is particularly valuable in practice, as it helps developers assess the credibility of warnings and reduces the effort required for manual vulnerability analysis.

5.3. Why do a significant number of negative samples get misclassified as positive by CoTVD?

In RQ1, we found that the CoTVD based on GPT-4o-128K classified many negative samples as positive samples, resulting in low precision (39.09%). We manually analyzed the detection results of GPT-4o-128K-based CoTVD on false positive samples and found that CoTVD would label any sample with safety risks as a positive sample, the security risks here are not limited to traditional security risks such as vulnerabilities, but also include risks such as information leakage. We summarize these risks into four categories:

#1 Risks related to print statements: If a print statement outputs sensitive data, CoTVD considers it a potential information leak and thus labels the sample as a positive sample.

#2 Improper error handling: If error messages are output using printf and then exit is called, CoTVD considers this not a good way to handle errors and would suggest returning an error code or using exceptions instead, marking the sample as a positive sample.

```
printf("out of memory");
exit(U_MEMORY_ALLOCATION_ERROR );
```

Table 6
Inference latency of open-source LLMs under CoTVD.

Model	Parameters	Avg. inference time (s/sample)
Qwen2-72B-Instruct-T	72B	5.8
DeepSeek-67B-T	67B	5.2
Deployment hardware	4 × NVIDIA A80 (80 GB)	

CoTVD: The error handling uses `printf` to output an error message and then calls `exit`. While this exits the program on failure, it might not be ideal in all contexts, especially in a library or shared component where exiting the entire process might not be appropriate. Consider returning an error code or using exceptions instead.

When error handling prints sensitive information, CoTVD considers it a risk of privacy leakage and labels the sample as a positive sample.

```
return apr_psprintf(cmd→pool, "ModSecurity: Rule phases cannot be
updated via SecRuleUpdateActionById.");
```

CoTVD: Error Handling: The function returns error messages directly, which could potentially leak sensitive information if not handled correctly elsewhere.
Label: 1 (Vulnerable)

When CoTVD believes that there should be error checking to improve code robustness, for example, in the following example, CoTVD believes that the code lacks a check for whether the `fp` pointer is null, CoTVD will mark the sample as a positive sample.

```
static void cooked (struct parse *pcmd, FILE *fp){
    rawmode = 0 ;
    (void)fprintf ( fp , "Output set to cooked" );
    return ;
}
```

CoTVD: Potential Issue: If `fp` is NULL or not properly opened, `fprintf` could fail. However, the code lacks error handling for this scenario. It would be prudent to check if `fp` is valid before attempting to write to it.
Label: 1 (Vulnerable)

#3 Security dependent on unknown external information: In some cases, the security behavior of a function depends on external information or implementation details that are not available in the extracted dependency slice, such as configuration values, environment variables, or the internal logic of external or unresolved function calls. When such information is missing, CoTVD may conservatively flag the corresponding code as potentially risky, even though the sample is labeled as non-vulnerable in the dataset. These cases do not necessarily indicate actual security vulnerabilities, but rather reflect the uncertainty introduced by incomplete contextual information. As a result, they are categorized as false positives in our analysis, highlighting a trade-off between vulnerability sensitivity and contextual completeness.

```
printf(buf, "affected rows: %s", llstr(affected_rows, buff2))
dynstr_append(ds, buf);
if(info){
    dynstr_append(ds, "info:");
    dynstr_append(ds, info);
    dynstr_append_mem(ds, "", 1);
}
```

CoTVD: The implementation details of `llstr` and `dynstr_append` are crucial. Any issues in these functions could also lead to vulnerabilities, especially if they do not adequately handle string sizes.
Label: 1 (Vulnerable)

When unknown variables or constants appear in the context, CoTVD prompts to check the definitions of these variables or constants to ensure that related operations will not cause problems, and the sample will be labeled as a positive sample.

```
if(!memcmp(subformat+4, (const uint8_t[]){
    FF_MEDIASUBTYPE_BASE_GUID }, 12)){
    par→codec_tag = AV_RL32(subformat);
    par→codec_id = ff_wav_codec_get_id(par→codec_tag,
    par→bits_per_coded_sample);
}
```

CoTVD: The line `memcmp(subformat + 4, (const uint8_t[]) FF_MEDIASUBTYPE_BASE_GUID, 12)` compares part of the subformat with a constant array. Ensure that subformat is well-defined and has enough allocated space before performing this operation to prevent out-of-bounds access.
Label: 1 (Vulnerable)

When an unknown array occurs, CoTVD recommends checking if the index exceeds the array's limit and marking the sample as a positive sample.

```
ret = ipvideo_decode_block[opcode](s, frame);
ret = ipvideo_decode_block16[opcode](s, frame);
```

CoTVD: Concern: Ensure that `opcode` is properly validated and within the bounds of the `ipvideo_decode_block` and `ipvideo_decode_block16` arrays. If `opcode` is out of bounds, it could lead to undefined behavior, potentially executing unintended code. Recommendation: Validate `opcode` before using it to index into the function pointer arrays.
Label: 1 (Vulnerable)

#4 Security dependent on runtime unknowns: In extremely rare cases (only once), CoTVD will consider that the execution of the program in a multi-threaded environment may lead to race conditions, and will label the sample as a positive sample.

```
enum nss_status _nss_nis_setaliasent(void){
    new_start = 1;
    if(oldkey != NULL){
        free(oldkey);
        oldkey = NULL;
        oldkeylen = 0 ;
    }
    return NSS_STATUS_SUCCESS;
}
```

CoTVD: The code snippet implies that `oldkey`, `oldkeylen`, and `new_start` are likely global or static variables, as they are not defined within the function. **If multiple threads access these variables**, ensure that all accesses are properly synchronized using the same lock to prevent race conditions.

Label: 1 (Vulnerable)

The above are the four types of risks that lead to false positive samples in CoTVD, although they do not directly cause vulnerabilities, they reflect potential security risks in code design. However, the ground truth provided by the dataset labels these samples as negative samples (no vulnerabilities). This also reflects the potential shortcomings of the automatic labeling method based on commits—the fact that developers have not modified the code does not necessarily mean there are no security risks. The security risks here may not be limited to the traditional security risks that lead to vulnerabilities, but may also include risks such as information leakage. At the same time, future work can further divide the dataset labels into more granular categories, such as no risk, potential risk, and confirmed vulnerabilities.

Next, we have “corrected” 459 false positive samples generated by CoTVD due to these reasons. These samples were marked as positive by CoTVD, and we adjusted them to negative samples. The “corrected” precision has improved to 86.94%. The F_1 score has increased from 0.530 to 0.907, which is consistent with our previous analysis that the model tends to mark samples with potential security risks as vulnerable, thereby increasing the false positive rate. The behavior of CoTVD producing false positives can be attributed to a high sensitivity to potential risks in the code rather than strict identification of explicit vulnerabilities. Under the guidance of CoTVD prompts, GPT-4o-128K often “over-detects”, marking code as vulnerable when there are inappropriate print statements, error handling, or unknown external information. This “over-detection” leads to an increase in the false positive rate, but from the perspective of security enhancement, CoTVD reveals more potential design flaws in the code.

6. Threats to validity

In designing and executing the experiment, we identified several factors that could impact CoTVD’s validity. These potential threats can be classified into internal validity threats, external validity threats, and construct validity threats.

6.1. Internal validity threats

Dataset Bias: Dataset Bias: The experiment only focuses on code samples involving C/C++ library function calls and has not been evaluated in other programming languages such as Java and Python, which exists limitations on the dataset. However, considering the similarity between programming languages and the universality of data dependency and control dependency across different programming languages, this threat is limited. *Furthermore, this work currently considers only C/C++ library API functions. In future work, we will extend CoTVD to cover other code elements that may lead to buffer overflow vulnerabilities, such as pointer, array, and arithmetic expressions.*

Prompt Design Bias: The prompts of CoTVD consist of five components, which are designed to guide the LLM in analyzing data dependencies and control dependencies that play a crucial role. However, there may be design issues that lead to inconsistent performance across different scenarios. We ensure the consistency of performance in various code contexts through ablation experiments and multiple iterations of prompt design, aiming to minimize the threat to effectiveness as much as possible.

Reliability of the Joern: In the experiment, we use Joern to extract data dependency and control dependency slices, so its reliability directly affects the performance of CoTVD. To mitigate this threat, we manually sample the outputs for inspection and compare them with other code parsing tools to ensure stability and accuracy.

6.2. External validity threats

External validity threats affect the general applicability of experimental results. The performance of CoTVD varies across different LLMs. From our experimental results, the CoTVD based on GPT-4-128K performs better than other models, but this may be due to the better adaptability of GPT-4o-128k to CoTVD. To mitigate this threat, we implemented CoTVD based on multiple common advanced LLMs in our experiments for a comprehensive evaluation.

6.3. Construct validity threats

Constructing validity threats and experimental methods and metrics are related to whether they can effectively evaluate model performance. Our evaluation relies on common metrics such as accuracy, recall rate, and F_1 score. However, these metrics may not fully capture the effectiveness of CoTVD in real-world development environments. To address this issue, we assess the application performance of CoTVD in real-world scenarios by inviting human experts to participate in experiments (RQ3).

7. Conclusion and future work

7.1. Conclusion

This research proposes CoTVD, a vulnerability detection and analysis method based on LLMs, which is the first attempt to use the reasoning ability of LLMs to analyze data dependencies and control dependencies for vulnerability detection. By utilizing the Chain-of-Thought, CoTVD achieves function-level vulnerability detection while providing detailed explanations of the potential causes of vulnerabilities. Experimental results show that the CoTVD based on GPT-4o-128K outperforms deep learning methods in terms of precision, recall rate, and F_1 score.

We also found that CoTVD shows a high sensitivity to potential risks or design flaws in the code, and here the risks are not limited to traditional vulnerabilities, but also include information leakage and so on. This sensitivity provides developers with more comprehensive protection, helping them to discover overlooked security issues early. However, we also found that the existing automated vulnerability dataset labeling methods may ignore labeling certain potential risks, especially those based on commit, because code that has not been modified in the commit may not be free of security risks. Finally, we also invited 10 human experts to detect 200 vulnerability samples, and with the help of CoTVD, each participant, on average, identified an additional 1.7 vulnerability samples and 1.1 lines of additional risk code, further proving the practical value of CoTVD.

7.2. Future work

Based on the findings and challenges identified in our current experiments, we propose the following directions for future research. First, the quality of dataset labeling has a significant impact on model performance. Future research can consider more granular dataset label granularity, such as no risk, potential risk, confirmed vulnerabilities, and create more comprehensive dataset labels. Second, the design of prompts is essential to the analysis results produced by LLMs. Future work could investigate more advanced prompt-construction methods, potentially incorporating dynamic adjustments to prompt content to meet the varying needs of different code segments and to identify vulnerabilities across different risk levels.

CRedit authorship contribution statement

Yinan Chen: Methodology. **Xiangping Chen:** Methodology. **Yuan Huang:** Software. **Changlin Yang:** Data curation. **Lei Yun:** Methodology.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

The data that support the findings of this study are openly available at <https://github.com/AFE23-u/CoTVD>.

References

- [1] B. Steenhoeck, M.M. Rahman, R. Jiles, W. Le, An empirical study of deep learning models for vulnerability detection, in: Proceedings of the 45th International Conference on Software Engineering, ICSE '23, IEEE Press, 2023, pp. 2237–2248, <http://dx.doi.org/10.1109/ICSE48619.2023.00188>.
- [2] B. Steenhoeck, H. Gao, W. Le, Dataflow analysis-inspired deep learning for efficient vulnerability detection, in: Proceedings of the IEEE/ACM 46th International Conference on Software Engineering, ICSE '24, Association for Computing Machinery, New York, NY, USA, 2024, <http://dx.doi.org/10.1145/3597503.3623345>.
- [3] Z. Li, N. Wang, D. Zou, Y. Li, R. Zhang, S. Xu, C. Zhang, H. Jin, On the effectiveness of function-level vulnerability detectors for inter-procedural vulnerabilities, in: 2024 IEEE/ACM 46th International Conference on Software Engineering, ICSE, 2024, pp. 1935–1946, <http://dx.doi.org/10.1145/3597503.3639218>.
- [4] E. Iannone, G. Sellitto, E. Iaccarino, F. Ferrucci, A. De Lucia, F. Palomba, Early and realistic exploitability prediction of just-disclosed software vulnerabilities: How reliable can it be? ACM Trans. Softw. Eng. Methodol. 33 (6) (2024) <http://dx.doi.org/10.1145/3654443>.
- [5] J. Gao, Y. Jiang, Z. Liu, X. Yang, C. Wang, X. Jiao, Z. Yang, J. Sun, Semantic learning and emulation based cross-platform binary vulnerability seeker, IEEE Trans. Softw. Eng. 47 (11) (2021) 2575–2589, <http://dx.doi.org/10.1109/TSE.2019.2956932>.
- [6] J. Guo, J. Cheng, J. Cleland-Huang, Semantically enhanced software traceability using deep learning techniques, in: 2017 IEEE/ACM 39th International Conference on Software Engineering, ICSE, 2017, pp. 3–14, <http://dx.doi.org/10.1109/ICSE.2017.9>.
- [7] M. White, M. Tufano, C. Vendome, D. Poshvanyk, Deep learning code fragments for code clone detection, in: 2016 31st IEEE/ACM International Conference on Automated Software Engineering, ASE, 2016, pp. 87–98.
- [8] E.C.R. Shin, D. Song, R. Moazzezi, Recognizing functions in binaries with neural networks, in: Proceedings of the 24th USENIX Conference on Security Symposium, SEC '15, USENIX Association, USA, 2015, pp. 611–626.
- [9] G. Lin, J. Zhang, W. Luo, L. Pan, Y. Xiang, POSTER: Vulnerability discovery with function representation learning from unlabeled projects, in: Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security, CCS '17, Association for Computing Machinery, New York, NY, USA, 2017, pp. 2539–2541, <http://dx.doi.org/10.1145/3133956.3138840>.
- [10] S. Hochreiter, J. Schmidhuber, Long short-term memory, Neural Comput. 9 (8) (1997) 1735–1780, <http://dx.doi.org/10.1162/neco.1997.9.8.1735>.
- [11] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A.N. Gomez, L. Kaiser, I. Polosukhin, Attention is all you need, in: Proceedings of the 31st International Conference on Neural Information Processing Systems, NIPS '17, Curran Associates Inc., Red Hook, NY, USA, 2017, pp. 6000–6010.
- [12] Y. Zhou, S. Liu, J. Siow, X. Du, Y. Liu, Devign: effective vulnerability identification by learning comprehensive program semantics via graph neural networks, in: Proceedings of the 33rd International Conference on Neural Information Processing Systems, Curran Associates Inc., Red Hook, NY, USA, 2019.
- [13] S. Chakraborty, R. Krishna, Y. Ding, B. Ray, Deep learning based vulnerability detection: Are we there yet? IEEE Trans. Softw. Eng. 48 (9) (2022) 3280–3296, <http://dx.doi.org/10.1109/TSE.2021.3087402>.
- [14] ChatGPT, 2024, URL <https://openai.com/index/chatgpt/>.
- [15] J. Wei, X. Wang, D. Schuurmans, M. Bosma, B. Ichter, F. Xia, E.H. Chi, Q.V. Le, D. Zhou, Chain-of-thought prompting elicits reasoning in large language models, in: Proceedings of the 36th International Conference on Neural Information Processing Systems, NIPS '22, Curran Associates Inc., Red Hook, NY, USA, 2024.
- [16] J. Ferrante, K.J. Ottenstein, J.D. Warren, The program dependence graph and its use in optimization, ACM Trans. Program. Lang. Syst. 9 (3) (1987) 319–349, <http://dx.doi.org/10.1145/24039.24041>.
- [17] Joern, 2024, <https://joern.io>. (Accessed 26 October 2024).
- [18] Gemini, 2024, <https://deepmind.google/technologies/gemini/pro/>. (Accessed 26 October 2024).
- [19] Claude, 2024, <https://www.anthropic.com/claude/sonnet>. (Accessed 26 October 2024).
- [20] Llama, 2024, https://www.llama.com/llama3_1/. (Accessed 26 October 2024).
- [21] Qwen2, 2024, <https://qwenlm.github.io/zh/blog/qwen2/>. (Accessed 26 October 2024).
- [22] DeepSeek, 2024, <https://www.deepseek.com>. (Accessed 26 October 2024).
- [23] Flawfinder, 2024, URL <https://d Wheeler.com/flawfinder/>.
- [24] Checkmarx, 2024, URL <https://checkmarx.com>.
- [25] Y. Shin, A. Meneely, L. Williams, J.A. Osborne, Evaluating complexity, code churn, and developer activity metrics as indicators of software vulnerabilities, IEEE Trans. Softw. Eng. 37 (6) (2011) 772–787, <http://dx.doi.org/10.1109/TSE.2010.81>.
- [26] Z. Li, D. Zou, S. Xu, X. Ou, H. Jin, S. Wang, Z. Deng, Y. Zhong, VulDeePecker: A deep learning-based system for vulnerability detection, 2018.
- [27] Z. Li, D. Zou, S. Xu, H. Jin, Y. Zhu, Z. Chen, SySeVR: A framework for using deep learning to detect software vulnerabilities, IEEE Trans. Dependable Secur. Comput. 19 (4) (2022) 2244–2258, <http://dx.doi.org/10.1109/TDSC.2021.3051525>.
- [28] Y. Fan, C. Wan, C. Fu, L. Han, H. Xu, VDoTR: Vulnerability detection based on tensor representation of comprehensive code graphs, Comput. Secur. 130 (C) (2023) <http://dx.doi.org/10.1016/j.cose.2023.103247>.
- [29] A. Bagheri, P. Hegedűs, A comparison of different source code representation methods for vulnerability prediction in Python, in: A.C.R. Paiva, A.R. Cavalli, P. Ventura Martins, R. Pérez-Castillo (Eds.), Quality of Information and Communications Technology, Springer International Publishing, Cham, 2021, pp. 267–281.
- [30] B. Steenhoeck, M.M. Rahman, R. Jiles, W. Le, An empirical study of deep learning models for vulnerability detection, 2023, URL <https://arxiv.org/abs/2212.08109>, arXiv:2212.08109.
- [31] S. Kim, J. Choi, M.E. Ahmed, S. Nepal, H. Kim, VulDeBERT: A vulnerability detection system using BERT, in: 2022 IEEE International Symposium on Software Reliability Engineering Workshops, ISSREW, 2022, pp. 69–74, <http://dx.doi.org/10.1109/ISSREW55968.2022.00042>.
- [32] I. Kaloutsoglou, M. Siavvas, A. Ampatzoglou, D. Kehagias, A. Chatzigeorgiou, Transfer learning for software vulnerability prediction using transformer models, J. Syst. Softw. 227 (C) (2025) <http://dx.doi.org/10.1016/j.jss.2025.112448>.
- [33] W. Ma, S. Liu, Z. Lin, W. Wang, Q. Hu, Y. Liu, C. Zhang, L. Nie, L. Li, Y. Liu, LMs: Understanding code syntax and semantics for code analysis, 2024, URL <https://arxiv.org/abs/2305.12138>, arXiv:2305.12138.
- [34] C. Wang, Y. Yang, C. Gao, Y. Peng, H. Zhang, M.R. Lyu, No more fine-tuning? an experimental evaluation of prompt tuning in code intelligence, in: Proceedings of the 30th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering, in: ESEC/FSE 2022, Association for Computing Machinery, New York, NY, USA, 2022, pp. 382–394, <http://dx.doi.org/10.1145/3540250.3549113>.
- [35] C. Wang, Y. Yang, C. Gao, Y. Peng, H. Zhang, M.R. Lyu, Prompt tuning in code intelligence: An experimental evaluation, IEEE Trans. Softw. Eng. 49 (11) (2023) 4869–4885, <http://dx.doi.org/10.1109/TSE.2023.3313881>.
- [36] Y. Wang, W. Wang, S. Joty, S.C. Hoi, CodeT5: Identifier-aware unified pre-trained encoder-decoder models for code understanding and generation, in: M.-F. Moens, X. Huang, L. Specia, S.W.-t. Yih (Eds.), Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing, Association for Computational Linguistics, Online and Punta Cana, Dominican Republic, 2021, pp. 8696–8708, <http://dx.doi.org/10.18653/v1/2021.emnlp-main.685>, URL <https://aclanthology.org/2021.emnlp-main.685>.
- [37] W. Ahmad, S. Chakraborty, B. Ray, K.-W. Chang, Unified pre-training for program understanding and generation, in: K. Toutanova, A. Rumshisky, L. Zettlemoyer, D. Hakkani-Tur, I. Beltagy, S. Bethard, R. Cotterell, T. Chakraborty, Y. Zhou (Eds.), Proceedings of the 2021 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Association for Computational Linguistics, 2021, pp. 2655–2668, <http://dx.doi.org/10.18653/v1/2021.naacl-main.211>, Online, URL <https://aclanthology.org/2021.naacl-main.211>.
- [38] Z. Feng, D. Guo, D. Tang, N. Duan, X. Feng, M. Gong, L. Shou, B. Qin, T. Liu, D. Jiang, M. Zhou, CodeBERT: A pre-trained model for programming and natural languages, in: T. Cohn, Y. He, Y. Liu (Eds.), Findings of the Association for Computational Linguistics, EMNLP 2020, Association for Computational Linguistics, Online, 2020, pp. 1536–1547, <http://dx.doi.org/10.18653/v1/2020.findings-emnlp.139>, URL <https://aclanthology.org/2020.findings-emnlp.139>.
- [39] C. Chen, J. Su, J. Chen, Y. Wang, T. Bi, J. Yu, Y. Wang, X. Lin, T. Chen, Z. Zheng, When ChatGPT meets smart contract vulnerability detection: How far are we? 2024, URL <https://arxiv.org/abs/2309.05520>, arXiv:2309.05520.
- [40] Z. Fan, X. Gao, M. Mirchev, A. Roychoudhury, S.H. Tan, Automated repair of programs from large language models, in: Proceedings of the 45th International Conference on Software Engineering, ICSE '23, IEEE Press, 2023, pp. 1469–1481, <http://dx.doi.org/10.1109/ICSE48619.2023.00128>.
- [41] Y. Zhang, Detecting code comment inconsistencies using LLM and program analysis, in: Companion Proceedings of the 32nd ACM International Conference on the Foundations of Software Engineering, in: FSE 2024, Association for Computing Machinery, New York, NY, USA, 2024, pp. 683–685, <http://dx.doi.org/10.1145/3663529.3664458>.

- [42] J. Xu, Z. Cui, Y. Zhao, X. Zhang, S. He, P. He, L. Li, Y. Kang, Q. Lin, Y. Dang, S. Rajmohan, D. Zhang, UniLog: Automatic logging via LLM and in-context learning, in: Proceedings of the IEEE/ACM 46th International Conference on Software Engineering, ICSE '24, Association for Computing Machinery, New York, NY, USA, 2024, <http://dx.doi.org/10.1145/3597503.3623326>.
- [43] Y. Cheng, J. Chen, Q. Huang, Z. Xing, X. Xu, Q. Lu, Prompt sapper: A LLM-empowered production tool for building AI chains, ACM Trans. Softw. Eng. Methodol. 33 (5) (2024) <http://dx.doi.org/10.1145/3638247>.
- [44] O. Rubin, J. Herzig, J. Berant, Learning to retrieve prompts for in-context learning, in: M. Carpuat, M.-C. de Marneffe, I.V. Meza Ruiz (Eds.), Proceedings of the 2022 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Association for Computational Linguistics, Seattle, United States, 2022, pp. 2655–2671, <http://dx.doi.org/10.18653/v1/2022.naacl-main.191>, URL <https://aclanthology.org/2022.naacl-main.191>.
- [45] N. Nashid, M. Sintaha, A. Mesbah, Retrieval-based prompt selection for code-related few-shot learning, in: 2023 IEEE/ACM 45th International Conference on Software Engineering, ICSE, 2023, pp. 2450–2462, <http://dx.doi.org/10.1109/ICSE48619.2023.00205>.
- [46] Z. Shen, Z. Liu, J. Qin, M. Savvides, K.-T. Cheng, Partial is better than all: Revisiting fine-tuning strategy for few-shot learning, in: Proceedings of the AAAI Conference on Artificial Intelligence, vol. 35, (11) 2021, pp. 9594–9602, <http://dx.doi.org/10.1609/aaai.v35i11.17155>, URL <https://ojs.aaai.org/index.php/AAAI/article/view/17155>.
- [47] T. Schick, H. Schütze, Exploiting cloze-questions for few-shot text classification and natural language inference, in: P. Merlo, J. Tiedemann, R. Tsarfaty (Eds.), Proceedings of the 16th Conference of the European Chapter of the Association for Computational Linguistics: Main Volume, Association for Computational Linguistics, Online, 2021, pp. 255–269, <http://dx.doi.org/10.18653/v1/2021.eacl-main.20>, URL <https://aclanthology.org/2021.eacl-main.20>.
- [48] X.L. Li, P. Liang, Prefix-tuning: Optimizing continuous prompts for generation, in: C. Zong, F. Xia, W. Li, R. Navigli (Eds.), Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics and the 11th International Joint Conference on Natural Language Processing (Volume 1: Long Papers), Association for Computational Linguistics, Online, 2021, pp. 4582–4597, <http://dx.doi.org/10.18653/v1/2021.acl-long.353>, URL <https://aclanthology.org/2021.acl-long.353>.
- [49] Y. Guo, H. Shi, A. Kumar, K. Grauman, T. Rosing, R. Feris, SpotTune: Transfer learning through adaptive fine-tuning, in: 2019 IEEE/CVF Conference on Computer Vision and Pattern Recognition, CVPR, 2019, pp. 4800–4809, <http://dx.doi.org/10.1109/CVPR.2019.00494>.
- [50] Y. Peng, C. Wang, W. Wang, C. Gao, M.R. Lyu, Generative type inference for Python, in: 2023 38th IEEE/ACM International Conference on Automated Software Engineering, ASE, 2023, pp. 988–999, <http://dx.doi.org/10.1109/ASE56229.2023.00031>.
- [51] QEMU, 2024, <https://www.qemu.org>. (Accessed 26 October 2024).
- [52] Wireshark, 2024, <https://www.wireshark.org>. (Accessed 26 October 2024).
- [53] Ffmpeg, 2024, <https://ffmpeg.org>. (Accessed 26 October 2024).
- [54] J. Renze, S. Guven, A systematic study of the impact of sampling temperature on large language model performance, 2024, arXiv preprint [arXiv:2402.05201](https://arxiv.org/abs/2402.05201).
- [55] H. Touvron, T. Lavril, G. Izacard, X. Martinet, M.-A. Lachaux, T. Lacroix, B. Rozière, N. Goyal, E. Hambro, F. Azhar, A. Rodriguez, A. Joulin, E. Grave, G. Lample, LLaMA: Open and efficient foundation language models, 2023, URL <https://arxiv.org/abs/2302.13971>, arXiv:2302.13971.
- [56] A. Yang, B. Yang, B. Hui, B. Zheng, B. Yu, C. Zhou, C. Li, C. Li, D. Liu, F. Huang, G. Dong, H. Wei, H. Lin, J. Tang, J. Wang, J. Yang, J. Tu, J. Zhang, J. Ma, J. Yang, J. Xu, J. Zhou, J. Bai, J. He, J. Lin, K. Dang, K. Lu, K. Chen, K. Yang, M. Li, M. Xue, N. Ni, P. Zhang, P. Wang, R. Peng, R. Men, R. Gao, R. Lin, S. Wang, S. Bai, S. Tan, T. Zhu, T. Li, T. Liu, W. Ge, X. Deng, X. Zhou, X. Ren, X. Zhang, X. Wei, X. Ren, X. Liu, Y. Fan, Y. Yao, Y. Zhang, Y. Wan, Y. Chu, Y. Liu, Z. Cui, Z. Zhang, Z. Guo, Z. Fan, Qwen2 technical report, 2024, URL <https://arxiv.org/abs/2407.10671>, arXiv:2407.10671.
- [57] DeepSeek-AI, A. Liu, B. Feng, B. Xue, B. Wang, B. Wu, C. Lu, C. Zhao, C. Deng, C. Zhang, C. Ruan, D. Dai, D. Guo, D. Yang, D. Chen, D. Ji, E. Li, F. Lin, F. Dai, F. Luo, G. Hao, G. Chen, G. Li, H. Zhang, H. Bao, H. Xu, H. Wang, H. Zhang, H. Ding, H. Xin, H. Gao, H. Li, H. Qu, J.L. Cai, J. Liang, J. Guo, J. Ni, J. Li, J. Wang, J. Chen, J. Chen, J. Yuan, J. Qiu, J. Li, J. Song, K. Dong, K. Hu, K. Gao, K. Guan, K. Huang, K. Yu, L. Wang, L. Zhang, L. Xu, L. Xia, L. Zhao, L. Wang, L. Zhang, M. Li, M. Wang, M. Zhang, M. Zhang, M. Tang, M. Li, N. Tian, P. Huang, P. Wang, P. Zhang, Q. Wang, Q. Zhu, Q. Chen, Q. Du, R.J. Chen, R.L. Jin, R. Ge, R. Zhang, R. Pan, R. Wang, R. Xu, R. Zhang, R. Chen, S.S. Li, S. Lu, S. Zhou, S. Chen, S. Wu, S. Ye, S. Ye, S. Ma, S. Wang, S. Zhou, S. Yu, S. Zhou, S. Pan, T. Wang, T. Yun, T. Pei, T. Sun, W.L. Xiao, W. Zeng, W. Zhao, W. An, W. Liu, W. Liang, W. Gao, W. Yu, W. Zhang, X.Q. Li, X. Jin, X. Wang, X. Bi, X. Liu, X. Wang, X. Shen, X. Chen, X. Zhang, X. Chen, X. Nie, X. Sun, X. Wang, X. Cheng, X. Liu, X. Xie, X. Liu, X. Yu, X. Song, X. Shan, X. Zhou, X. Yang, X. Li, X. Su, X. Lin, Y.K. Li, Y.Q. Wang, Y.X. Wei, Y.X. Zhu, Y. Zhang, Y. Xu, Y. Xu, Y. Huang, Y. Li, Y. Zhao, Y. Sun, Y. Li, Y. Wang, Y. Yu, Y. Zheng, Y. Zhang, Y. Shi, Y. Xiong, Y. He, Y. Tang, Y. Piao, Y. Wang, Y. Tan, Y. Ma, Y. Liu, Y. Guo, Y. Wu, Y. Ou, Y. Zhu, Y. Wang, Y. Gong, Y. Zou, Y. He, Y. Zha, Y. Xiong, Y. Ma, Y. Yan, Y. Luo, Y. You, Y. Liu, Y. Zhou, Z.F. Wu, Z.Z. Ren, Z. Ren, Z. Sha, Z. Fu, Z. Xu, Z. Huang, Z. Zhang, Z. Xie, Z. Zhang, Z. Hao, Z. Gou, Z. Ma, Z. Yan, Z. Shao, Z. Xu, Z. Wu, Z. Zhang, Z. Li, Z. Gu, Z. Zhu, Z. Liu, Z. Li, Z. Xie, Z. Song, Z. Gao, Z. Pan, DeepSeek-V3 technical report, 2025, URL <https://arxiv.org/abs/2412.19437>, arXiv:2412.19437.
- [58] M. Pendleton, R. Garcia-Lebron, J.-H. Cho, S. Xu, A survey on systems security metrics, ACM Comput. Surv. 49 (4) (2016) <http://dx.doi.org/10.1145/3005714>.